

Jekyll

Notes: What It Is, Why It's Popular, and How It Works

1. What Is Jekyll?

Jekyll is a free, open-source **static site generator** written in Ruby. Instead of building pages dynamically on every request (like WordPress or a database-backed CMS), Jekyll takes raw text files — written in Markdown, HTML, and Liquid templates — and compiles them, once, into a complete set of static HTML, CSS, and JavaScript files. Those static files can then be hosted anywhere: GitHub Pages, Netlify, Vercel, S3, or any basic web server.

It was created in 2008 by Tom Preston-Werner, a co-founder of GitHub, which is a big part of why Jekyll and GitHub Pages are so tightly linked — GitHub Pages runs Jekyll natively, so you can push a Jekyll repo straight to GitHub and get a live site with zero server setup.

2. Why Is Jekyll Popular?

1. Free native GitHub Pages hosting

GitHub Pages builds and serves Jekyll sites automatically with no extra configuration, no server costs, and no third-party build pipeline. This alone drove huge adoption among developers who already live on GitHub.

2. Speed and simplicity

Because output is static HTML, pages load fast, there's no database to manage, and there's far less that can break or be hacked compared to dynamic CMS platforms.

3. Markdown-first writing

Content is written in plain Markdown files instead of a database or complex CMS interface, which developers and technical writers find faster and more version-control-friendly.

4. Git-based workflow

Since a Jekyll site is just a folder of text files, the entire site history, content, and design live in Git. Every change is tracked, reviewable, and revertible like code.

5. No server-side runtime needed

There's no PHP, no Node server, no database to secure or patch. Static files mean drastically smaller attack surface and near-zero maintenance overhead.

6. Strong ecosystem and themes

A large library of free and paid themes, plus hundreds of plugins for SEO, pagination, sitemaps, and RSS feeds, made it easy to get a polished blog or docs site running quickly.

7. Great fit for blogs, docs, and portfolios

Its built-in support for posts, front matter (metadata), and collections makes it a natural choice for blogs, documentation sites, and personal portfolios — exactly the sites developers build most.

3. How Jekyll Works (Under the Hood)

Jekyll follows a simple build pipeline. You write content and templates, and Jekyll's build process transforms them into a static site inside a `_site` folder, ready to deploy.

Typical Jekyll project structure:

```
your-site/
|-- _config.yml    # site-wide settings
|-- _posts/        # blog posts (dated Markdown files)
|-- _layouts/      # HTML templates (e.g. post.html, default.html)
|-- _includes/     # reusable snippets (header, footer, nav)
|-- _data/         # YAML/JSON/CSV data files
|-- assets/        # CSS, JS, images
|-- index.md       # homepage content
`-- _site/         # generated static output (build result)
```

The build process, step by step:

- 1 **Front matter parsing** — Every content file starts with a YAML block (between `---` lines) defining metadata like title, layout, date, and tags.
- 2 **Template rendering with Liquid** — Jekyll uses the Liquid templating language (also created at GitHub) to insert dynamic-looking content into static HTML — loops, conditionals, and variables like `{{ page.title }}` or `{% for post in site.posts %}`.
- 3 **Markdown conversion** — Content written in Markdown is converted into HTML using a Markdown engine (Kramdown by default).
- 4 **Layout application** — Each page/post is wrapped in the HTML layout specified in its front matter, and layouts can themselves be nested (e.g. a post layout inside a default site layout).
- 5 **Asset processing** — Sass/SCSS files are compiled to CSS, and other assets are copied over.
- 6 **Static output generation** — The final HTML, CSS, and JS files are written to the `_site` directory, ready to be uploaded or served as-is.

Because this entire process runs once at build time (not per visitor request), the resulting site requires no backend logic to serve — any static file host can run it.

4. Common Jekyll Commands

```
gem install jekyll bundler    # install Jekyll
jekyll new my-site            # scaffold a new site
cd my-site
bundle exec jekyll serve      # build + local dev server (localhost:4000)
jekyll build                   # build static output into _site/
```

5. Strengths vs. Limitations

Strengths	Limitations
-----------	-------------

Fast page loads (pre-built HTML)	No real-time dynamic content (search, comments need 3rd-party JS services)
Free hosting via GitHub Pages	Ruby-based tooling can be a hurdle on Windows
Minimal security surface (no DB/backend)	Large sites can have slower build times
Full version control via Git	Not ideal for frequently-changing, user-generated content
Simple Markdown-based authoring	Non-technical users find raw file/Git workflow harder than a GUI CMS

6. Where Jekyll Is Commonly Used

- Personal developer portfolios and blogs (your own site is a good example)
- Open-source project documentation sites
- GitHub Pages project sites (docs, changelogs, landing pages)
- Company engineering blogs where content is written by developers in Markdown

Note: Jekyll pioneered the modern "static site generator" category, but newer tools like Hugo (Go, faster builds), Eleventy (JS), Astro, and Next.js static export have since become popular alternatives — each trades off build speed, language ecosystem, and flexibility differently.